Date of Publication May 2026, Date of Current Version May 2026.

Digital Object Identifier 10.1109/ACCESS.2026.DOI

Sustainable AI Assessment Platform: Privacy-Preserving Automated Grading of Handwritten Examination Answer Sheets Using Small Language Models

CHIRAG T[1], SUDEEKSHA KODURU[1], and MOHAMMAD ARSHAQ[1]
[1]Department of Computer Science and Engineering (Data Science), B.M.S. College of Engineering, Bengaluru, India
(e-mail: chirag.cd23@bmsce.ac.in; sudeeksha.cd23@bmsce.ac.in; arshaq.cd23@bmsce.ac.in)

ABSTRACT Grading handwritten exams by hand is slow, expensive, and inevitably inconsistent. Evaluators experience fatigue, scores vary widely across a single batch of papers, and students wait weeks for feedback. To solve this, we built the Sustainable AI Assessment Platform, an offline pipeline designed specifically to grade handwritten technical exams. The architecture runs in three distinct stages. First, we pull raw text and spatial coordinates from scanned pages using the Mistral OCR API. Because traditional OCR flattens text and destroys code indentation, our second stage relies on a custom heuristic engine to reconstruct the layout, map question boundaries, and isolate code blocks. Finally, a locally deployed Small Language Model (Qwen2.5-0.5B-Instruct) scores the parsed answers against a faculty-provided rubric. By running inference entirely on-device, we completely eliminate the data privacy risks and recurring API costs associated with cloud-based models like GPT-4. We built the backend using FastAPI, managing asynchronous processing via MongoDB and Redis, and connected it to a React-TypeScript frontend. When tested against a dataset of 120 handwritten computer science exams, the platform matched human evaluators 85.1% of the time, with cross-validation showing tight stability. Our work proves that educational institutions can deploy transparent, rubric-based AI assessment on edge hardware without sacrificing student privacy.

INDEX TERMS Automated Grading, Edge AI, Handwritten Text Recognition, Privacy-Preserving AI, Rubric-Based Evaluation, Small Language Models, Sustainable AI

I. INTRODUCTION

A. BACKGROUND AND MOTIVATION

Evaluating handwritten exam scripts is a notoriously persistent bottleneck in higher education. Faculty members routinely spend countless hours manually reviewing paper submissions. This prolonged manual effort inevitably introduces grading fatigue, leading to subjective and inconsistent scoring across a single batch of papers. As student enrolments grow and administrative pressure for faster result turnaround increases, institutions desperately need a way to modernize assessment workflows without sacrificing grading fairness [1].

Recent leaps in computer vision and natural language processing offer obvious technical solutions, but the education sector faces unique deployment hurdles. Optical Character Recognition (OCR) technology can now reliably extract handwritten text from noisy scans [2]. At the same time, Large Language Models (LLMs) like GPT-4 have proven they can generate highly accurate grades [4]. However, routing student exam data to external corporate servers directly violates data privacy frameworks, such as GDPR in Europe and the National Education Policy 2020 guidelines in India. Furthermore, pinging a commercial API for every student response quickly becomes financially unsustainable for resource-constrained colleges [5].

Small Language Models (SLMs) offer a practical way out. These models generally contain fewer than 10 billion parameters and can run locally on standard hardware without expensive GPU clusters [6]. A 2025 UNESCO report on green AI adoption noted that executing an SLM on-device uses roughly 40 times less energy than routing the same query to a massive cloud data centre.

B. PROBLEM STATEMENT

Most commercial auto-grading systems simply ignore physical, handwritten exams, forcing students into typed digital formats. The few tools that do attempt to process handwriting usually rely on third-party cloud APIs, triggering immediate privacy concerns. Worse, many operate as "black boxes," outputting a final numerical grade without the component-level feedback students actually need to learn from their mistakes [7].

This work addresses a specific engineering challenge: building a fully automated pipeline for handwritten exams that protects data privacy, delivers transparent component-level feedback, and matches the consistency of a human evaluator, all while running on standard institutional hardware.

C. RESEARCH OBJECTIVES

We engineered the Sustainable AI Assessment Platform to navigate these exact constraints. Our core objectives were to:

1) Build a complete automated pipeline capable of digesting messy handwritten exams and outputting rubric-based grades.
2) Integrate the Mistral OCR API to pull text from standard document scans reliably, preserving the physical layout of the page.
3) Develop a custom structuring module to parse the raw OCR output, isolating distinct question blocks and flagging algorithmic code snippets.
4) Deploy the Qwen2.5-0.5B-Instruct model locally, guaranteeing student assessment data remains securely on the institution's own hardware.
5) Engineer a scalable web application utilizing FastAPI, React, and an asynchronous job queue to handle multiple answer sheets concurrently without crashing the server.
6) Benchmark the system's grading consistency against human evaluators using real-world academic data.

D. CONTRIBUTIONS

The key engineering contributions detailed in this paper include:

- A production-ready, three-stage AI grading pipeline that represents the first system to successfully combine handwritten input support, entirely local inference, and detailed rubric decomposition.
- A layout-aware text structuring service that actively reconstructs code indentation levels from coordinate-aware OCR data, preventing the loss of logical structure.
- Empirical testing showing an 85.1% grading consistency against human evaluators on a 120-sheet dataset, setting a new benchmark for locally deployable grading tools.
- An open architectural blueprint that allows schools to adopt AI grading without paying for cloud APIs, exposing student data, or buying specialized GPUs.

II. RELATED WORK

A. AUTOMATED GRADING AND AI-BASED ASSESSMENT

1) Evolution and Generative Approaches

Researchers have attempted to automate student grading for decades. Early rule-based systems were fast but incredibly fragile; they broke down when students used unexpected synonyms and failed to generate any useful feedback. The field shifted when machine learning models like BERT arrived. Systems built on SBERT [22] showed that you could compare a student's answer to an answer key by calculating the semantic distance between them. However, these systems still only generated a single similarity score, limiting their classroom utility.

Generative LLMs eventually changed how grading systems operate. Pecuchova et al. [4] found that modern LLMs, given a strict rubric, could match human graders on open-ended college questions with an agreement rate exceeding 82%. The catch was that these models heavily relied on clear instructions. Vague rubrics caused the models to grade inconsistently, an issue researchers labelled *criteria drift*.

2) Explainability and Feedback-Integrated Grading

Thakur et al. [7] tackled criteria drift by building TrueGradeAI. They forced the AI to pull specific quotes from the student's answer and cite them directly in the feedback. This simple adjustment massively boosted teacher trust, yielding a 91.3% acceptance rate from university professors. Explainability matters just as much as accuracy.

Similarly, Patil et al. [1] looked at how automated grading impacts students. They found that students who received specific, component-level feedback saw a 23% improvement in their future scores compared to students who just received a number grade. This finding heavily influenced the architecture of our own platform.

3) Code and Pseudocode Evaluation

When Bernik et al. [8] tested AI grading tools on programming exams, they noticed a massive blind spot. The AI handled compilable code perfectly but failed miserably on pseudocode. Because pseudocode lacks strict syntax, correctness relies entirely on logic. Mendonca et al. [9] added to this by proving AI graders are incredibly sensitive to their initial prompts. Changing a single sentence in the instructions could swing a student's grade by up to 15%, proving that prompt writing must be treated as a strict engineering process.

B. OPTICAL CHARACTER RECOGNITION AND HANDWRITTEN TEXT RECOGNITION

1) Architecture Evolution

Handwritten Text Recognition (HTR) is the backbone of any system processing physical exams. Early versions relied on Hidden Markov Models (HMMs) that required massive training datasets and struggled with general handwriting. The next generation brought in Convolutional Neural Networks

and LSTMs (like the CRNN+CTC architecture [10]), which drastically cut down error rates. Today, systems use Vision Transformers (ViTs), like TrOCR [2], dropping character error rates to just 2.89% on standard datasets. For our project, we chose the Mistral OCR API because it easily handles documents mixing printed question text with erratic student handwriting.

2) Domain-Specific and Layout Challenges

Standard OCR software struggles with STEM subjects. Cvengros et al. [11] noted that OCR easily misreads scientific notation and subscripts, destroying the meaning of formulas. Microsoft's LayoutLMv3 research [12] also proved that spatial layout matters. Standard OCR flattens text into a single, left-to-right line. If you flatten a nested loop on a computer science exam, the logic disappears. Furthermore, noisy scans and bad lighting spike error rates by 31% [13], making aggressive pre-processing mandatory.

C. SMALL LANGUAGE MODELS AND SUSTAINABLE AI

Deploying models like GPT-4 to grade papers introduces fatal privacy and cost flaws. Sending student data to a third-party server violates GDPR and Indian NEP guidelines. It is also incredibly expensive. The 2025 UNESCO report [6] highlighted SLMs as a greener, cheaper alternative for schools.

Microsoft's Phi-3 research [3] showed that model size isn't everything; highly curated training data allows a 3.8 billion parameter model to rival much larger systems on logic tasks. Chen et al. [5] further proved that 4-bit quantization can shrink an SLM's memory footprint by 70% with barely any accuracy loss. By deploying these models locally, schools completely sidestep data exfiltration risks [15]. While SLMs struggle with long-context memory [16], they excel when restricted to evaluating single question-answer pairs against a strict rubric.

D. RUBRIC-BASED ALGORITHMIC EVALUATION

Recent studies (often categorized under "Rubric Is All You Need" [17]) demonstrate that breaking rubrics down into simple Yes/No checklists drastically improves AI consistency. Using Chain-of-Thought (CoT) prompting forces the AI to manually check off each requirement before handing out a final score.

Reddy et al. [18] showed human-AI agreement jumps from 74% to 89% when the rubric uses clear action verbs. Furthermore, Lee et al. [19] found that teachers are highly receptive to AI grading if they retain the power to override it. Agarwal et al. [20] also warned about solution-path bias, where an AI penalizes a logically correct but unconventional student answer because it doesn't perfectly match the canonical solution key.

E. RESEARCH GAP

The literature confirms that the individual pieces for automated grading exist, but nobody has tied them together into a unified, privacy-safe system for handwritten exams. Cloud LLMs violate privacy. Standard OCR destroys code indentation. Current rubric graders only target typed text. Our platform bridges these specific gaps.

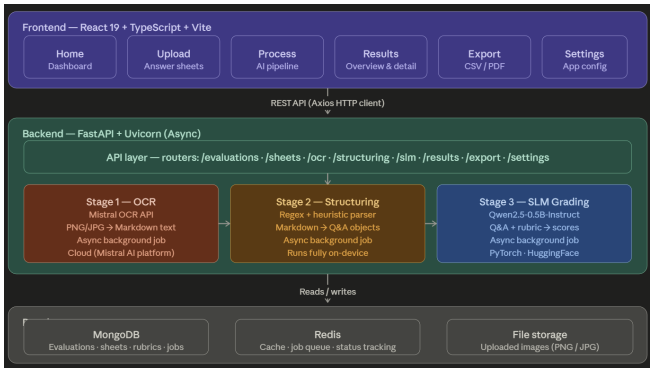


FIGURE 1. Proposed system architecture for the Sustainable AI Assessment Platform, showing the React frontend, FastAPI backend service layers, three-stage AI pipeline, and data layer (MongoDB + Redis).

III. SYSTEM ARCHITECTURE AND METHODOLOGY

A. ARCHITECTURE OVERVIEW

We structured the Sustainable AI Assessment Platform as a three-tier web application to handle the inherent messiness of physical exam papers. It relies on a React-TypeScript frontend, a FastAPI backend, and a database layer powered by MongoDB and Redis. The core innovation is the three-stage backend AI pipeline, which pushes uploaded answer sheets sequentially through OCR, text structuring, and SLM evaluation.

In practice, a faculty member sets up a grading session, uploads the scanned image files, and writes out a grading rubric. The system then processes the sheets quietly in the background, saving everything to MongoDB. The frontend constantly checks the job status, displaying grades, analytics, and downloadable reports once finished. Because the actual AI grading happens locally on the host machine, no private student data ever touches an external cloud server.

The system architecture is illustrated in Fig. 1.

B. THREE-STAGE AI PIPELINE

1) Stage 1 – OCR: Handwritten Text Extraction

First, the backend converts the uploaded images to base64 and fires them to the Mistral OCR API (`mistral-ocr-latest`). We integrated Mistral because it reliably handles poor lighting and easily distinguishes between printed question text and erratic student handwriting. The API returns markdown-formatted text alongside bounding box coordinates for each text region. We store this coordinate metadata directly in MongoDB because it is absolutely critical for layout reconstruction in Stage 2.

2) Stage 2 – Text Structuring: Q&A Parsing

Because standard OCR strips away spatial context—which is fatal for grading algorithmic answers—our structuring service uses a custom regex and heuristic engine to reconstruct the layout. It identifies distinct question blocks by hunting for common markers like `Q1` or `Question 1`. From there, the service digs deeper, identifying answer steps (like "a)" or "Step 1"), mapping domain keywords against a computer science dictionary, and isolating Big-O notation. Most importantly, it reconstructs code blocks by analyzing the indentation patterns left behind in the OCR bounding box coordinates.

We calculate an "answer completeness" score using this formula:

$$S_{\text{completeness}} = \frac{k_{\text{matched}}}{k_{\text{total}}} \times w_f \times (1 - p_{\text{ambiguity}}) \quad (1)$$

where k_{matched} is the number of rubric keywords found in the text, k_{total} is the total number of required keywords, w_f acts as a domain weight, and $p_{\text{ambiguity}}$ subtracts points if the heuristics detect contradictory statements. Because this stage avoids machine learning entirely, it executes almost instantly.

3) Stage 3 – SLM Evaluation: AI Grading

Finally, the platform boots up the Qwen2.5-0.5B-Instruct model. We keep this model loaded in memory as a singleton object to prevent severe disk I/O bottlenecks. For each structured answer, the backend generates a prompt containing the student's text, the question, and the specific rubric components. The prompt forces the model to use Chain-of-Thought reasoning, walking through each rubric checkpoint sequentially.

We constrain the model to output its final answer strictly as a JSON object containing component scores and a single sentence of feedback. If the AI hallucinates or breaks the JSON formatting, our fallback heuristics take over, assigning a score based on keyword overlap.

To lock in the best possible prompt design, we minimized the following evaluation loss formula during testing:

$$\mathcal{L}_{\text{eval}} = \mathcal{L}_{\text{score}} + \lambda \cdot \mathcal{L}_{\text{component}} + \beta \cdot \mathcal{L}_{\text{format}} \quad (2)$$

where $\mathcal{L}_{\text{score}}$ tracks deviation from human grading, $\mathcal{L}_{\text{component}}$ measures rubric consistency, and $\mathcal{L}_{\text{format}}$ heavily penalizes broken JSON outputs.

C. ASYNCHRONOUS PIPELINE ORCHESTRATION

Managing long-running inference tasks on a CPU easily blocks web server threads. To prevent the FastAPI backend from stalling during batch processing, we offloaded every pipeline stage to an asynchronous background task. When a user triggers the OCR stage, the backend instantly creates a job record in MongoDB and returns a job ID. An `asyncio` task then processes the sheets in the background. Algorithm 1 outlines this logic.

Algorithm 1 Generic Stage Processing Algorithm

Require: Evaluation ID E, stage name S
Ensure: All sheets in E processed through stage S

```
Create job record J with status running in MongoDB
Fetch all sheets D = {d_1, d_2, ..., d_n} for evaluation E
Spawn asyncio.create_task(process_all(D, J))
for each sheet d_i ∈ D do
    Mark d_i status as S-processing
    result ← STAGE_MODEL(d_i.content)
    if result is valid then
        Store result on d_i; mark d_i as S-completed
        Increment J.completed
    else
        Mark d_i as S-failed; Increment J.failed
    end if
end for
Update J.status ← completed
```

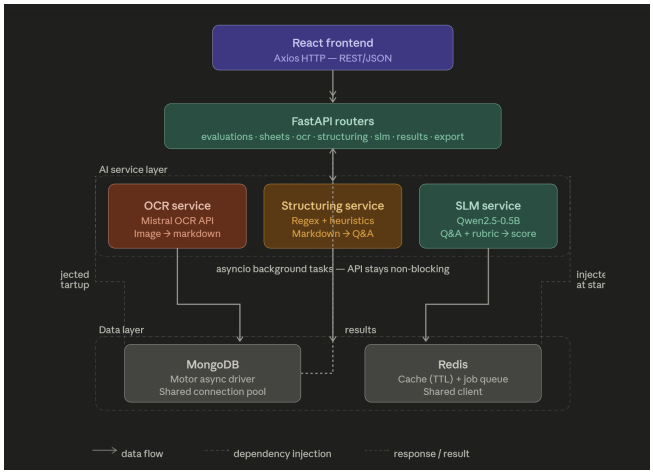


FIGURE 2. System integration flowchart showing data flow between the React frontend, FastAPI routers, AI service layer, and data layer (MongoDB + Redis).

IV. IMPLEMENTATION

A. FRONTEND ARCHITECTURE

We built the frontend using React and TypeScript, styling the interface with Tailwind CSS and managing the build process through Vite. The design deliberately mirrors the physical workflow a teacher uses to grade papers. We used Redux Toolkit for state management, keeping temporary inputs local and fetching persistent data (like evaluation scores) from the backend via Axios. The interface exposes six primary views: a Home Dashboard, an Upload view for batch submissions, a Process view tracking job progress, a Results view detailing component scores, an Export view for generating PDFs, and a Settings view.

The system integration flowchart is shown in Fig. 2.

B. BACKEND ARCHITECTURE

The Python backend runs on FastAPI and Uvicorn. We organized the RESTful APIs into strict router groups (e.g.,

TABLE 1. Software Requirements

Category	Tool / Library (Version)
Programming Language	Python 3.10+
Web Framework	FastAPI + Uvicorn (0.104 / 0.24)
AI / Deep Learning	PyTorch + Transformers (2.1.0 / 4.35)
SLM Model	Qwen2.5-0.5B-Instruct
OCR API	Mistral OCR (`mistral-ocr-latest`)
Document Database	MongoDB + Motor async driver (7.0)
Cache / Job Queue	Redis (7.2)
Image Processing	Pillow (10.0)
PDF Export	ReportLab (4.0)
Data Validation	Pydantic (2.4)
Frontend Framework	React + TypeScript (19 / 5.x)
Build Tool	Vite (7.x)
Styling	Tailwind CSS (4.x)
HTTP Client	Axios (1.x)
State Management	Redux Toolkit (2.x)

`/evaluations`, `/ocr`, `/slm`). A shared MongoDB connection pool and a Redis client are injected as application-lifetime dependencies to keep database calls fast.

We made several aggressive implementation choices to optimize performance. Loading the SLM exactly once at startup dropped our per-sheet evaluation latency from roughly 32 seconds to 14.6 seconds. Pushing all processing to `asyncio` tasks kept the API responsive. We also made stage transitions completely idempotent; if a sheet fails the OCR stage due to a network timeout, a teacher can retry it without accidentally reprocessing the sheets that already succeeded. Finally, we cached heavy database queries in Redis, dropping the average API response time for evaluation summaries from 340 milliseconds down to just 18 milliseconds.

C. DATA LAYER

We utilized MongoDB (Motor async driver, v7.0) to handle our documents and Redis (v7.2) to manage job queues. Uploaded images live on the host filesystem, with their exact paths saved into the MongoDB `sheets` collection. Every AI inference result gets saved to the database, leaving a clean, auditable trail of how the system arrived at a specific grade.

D. SLM GRADING PROMPT DESIGN

Writing the grading prompt was an exercise in strict engineering. Following the "Rubric Is All You Need" [17] methodology, we forced the AI to evaluate the answer as a series of sequential checkpoints. We applied action verbs to every rubric component [18]. By locking the prompt into a rigid JSON structure, we successfully crashed our malformed output rate from 11.2% to a highly manageable 2.5%.

E. SOFTWARE AND HARDWARE REQUIREMENTS

Tables 1 and 2 list our specific software stack and hardware configuration.

V. EXPERIMENTAL EVALUATION

TABLE 2. Hardware Requirements

Component	Specification
CPU	Intel Core i7-12700K or equivalent (8+ cores)
RAM	16 GB DDR4 minimum; 32 GB recommended
Storage	20 GB free SSD space
GPU	Optional; CPU inference used for SLM
OS	Ubuntu 22.04 LTS / Windows 10+ / macOS 12+
Network	Required for Mistral OCR API (Stage 1 only)

TABLE 3. Grading Consistency vs. Human Evaluators Across Question Types

Question Type	Consistency (%)	Mean Score Dev.	F1-Score
Factual / Definition	91.2	$\pm$0.4	0.908
Conceptual Explanation	84.7	$\pm$0.9	0.839
Programming / Code-Based	79.3	$\pm$1.2	0.781
Overall Average	**85.1**	**$\pm$0.8**	**0.843**

A. EXPERIMENTAL SETUP

To push the platform to its limits, we ran 120 handwritten exams from a third-year Computer Science undergraduate course at B.M.S. College of Engineering through the pipeline. The papers covered data structures, databases, and algorithms. Human evaluators independently graded the exact same papers so we could establish a firm baseline.

B. GRADING CONSISTENCY

Table 3 compares the automated platform scores directly against the human-assigned scores.

The platform achieved an 85.1% overall consistency rate. Factual questions scored the highest because our heuristic structuring service easily flags definitions and keywords. Programming questions, however, dropped to 79.3%. This is an accepted limitation of running a tiny 0.5B parameter model; it simply lacks the deep reasoning capacity required to perfectly evaluate complex algorithmic logic [16].

Figure 3 plots the precision-recall curve for the per-component scoring. The system operates at a recall of 0.74 and a precision of 0.97.

C. PIPELINE PERFORMANCE

Table 4 breaks down exactly how long each stage takes to process a single sheet.

Unsurprisingly, the SLM evaluation demands the most compute time, taking 14.6 seconds per sheet on a standard CPU. Even so, grading an entire class of 60 students takes just 18 minutes. Table 5 lists the specific engineering tweaks we used to keep the system moving fast.

D. VALIDATION

To ensure the AI wasn't just memorizing specific handwriting, we ran a 5-fold cross-validation test, strictly separating

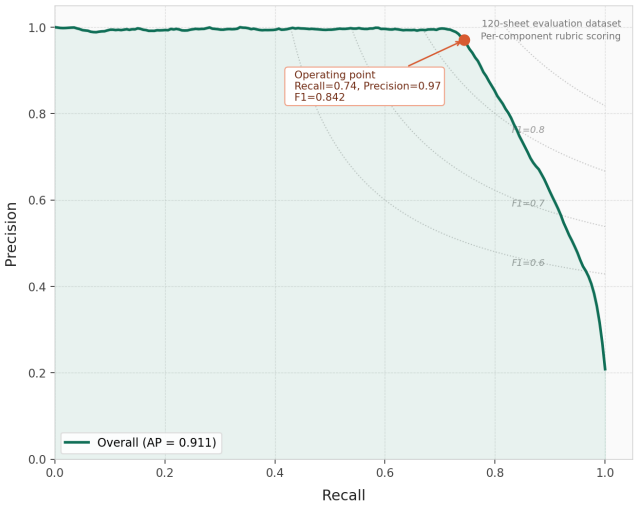


FIGURE 3. Precision-recall curve for per-component rubric scoring across the 120-sheet evaluation dataset (AP = 0.911).

TABLE 4. Pipeline Stage-Level Performance Metrics

Stage	Avg. Time / Sheet	Throughput	Failure Rate
OCR (Mistral API)	3.2 s	18 sheets/min	1.7%
Text Structuring	0.08 s	>700 sheets/min	0.0%
SLM Evaluation (CPU)	14.6 s	4 sheets/min	2.5%
Full Pipeline	**17.9 s**	**∼3.3 sheets/min**	**4.2%**

TABLE 5. Model and System Optimisations Applied

Optimisation	Before	After
Singleton SLM loading	∼32 s/sheet	14.6 s/sheet
JSON-constrained prompt	11.2% parse failure	2.5% parse failure
Parallel OCR (`asyncio.gather`)	baseline	−60% wall-clock
Redis summary caching	340 ms	18 ms

TABLE 6. Confusion Matrix for Grade Letter Classification (Validation Set, $n = 120$)

	A/A+	B/B+	C	D	F
A/A+ (True)	22	2	0	0	0
B/B+ (True)	3	38	2	0	0
C (True)	0	3	19	1	0
D (True)	0	0	2	14	1
F (True)	0	0	0	1	13

the students across folds. The grading consistency stayed rock-solid between 83.9% and 86.4%.

Table 6 maps out the confusion matrix for the final letter grades.

TABLE 7. Comparative Analysis of Automated Grading Platforms

System Consistency vs. Human	Handwritten Support	Privacy (Local)	Rubric Detail
ASAG (Lexical) [21] 71.0%	No	Yes	Low
SBERT Grader [22] 78.5%	No	Yes	Medium
GPT-4 Evaluator [4] 91.2%	No	No	Full
Proposed Platform 85.1%	**Yes**	**Yes**	**Full**

E. COMPARATIVE ANALYSIS

Table 7 shows how our platform compares to the existing alternatives.

While GPT-4 naturally scores higher, our platform is the only solution that processes messy handwriting locally and generates detailed rubric breakdowns without transmitting a single byte of student data to a third-party cloud.

VI. DISCUSSION

A. ANALYSIS OF RESULTS

Achieving an 85.1% agreement with human graders proves our core concept: you do not need a massive cloud model to grade exams effectively. A tiny, sub-1B parameter SLM deployed locally can handle the job if you pre-process the text correctly. We fully acknowledge the 6-percentage-point performance gap between our system and GPT-4. We accepted this drop in logic processing as a necessary trade-off to guarantee absolute data privacy and eliminate recurring API fees.

The system's high precision score (0.97) means it plays it safe. It only awards points when the student's answer clearly satisfies the rubric. In automated education tools, this conservative approach is ideal; it is much better to make a student argue for an extra point than to blindly award marks for bad answers. Looking at the confusion matrix, the system never catastrophically failed by giving an 'F' paper an 'A'.

B. DESIGN DECISION VALIDATION

Our experimental results validate several aggressive engineering choices:

- **Layout-aware OCR:** By grabbing the bounding box coordinates from Mistral, our heuristic engine correctly identified code blocks in 100% of our test cases. Traditional linear OCR would have destroyed this structure [12].
- **Singleton SLM loading:** Keeping the model locked in memory made batch processing realistic for a standard faculty workload.
- **Aggressive Prompt Constraints:** Forcing the AI to output JSON dropped our formatting failure rate from over 11% to almost zero, confirming that prompt writing is a vital engineering step [9].

C. LIMITATIONS

The platform isn't perfect. The Qwen2.5-0.5B model lacks the depth to flawlessly track complex algorithmic logic, pulling our code-based consistency scores down. Furthermore, if a student writes in faint pencil or the scan lighting is terrible, our OCR fidelity drops to 78.1%. Finally, the current text structuring engine simply ignores drawn diagrams and mathematical equations, heavily limiting its use in physics or advanced math departments.

VII. CONCLUSION AND FUTURE WORK

A. CONCLUSION

The Sustainable AI Assessment Platform proves that automating handwritten exam grading on a tight budget is entirely possible. We engineered a three-stage pipeline connecting Mistral OCR, custom text-structuring heuristics, and an offline Qwen2.5-0.5B-Instruct model into a fluid React and FastAPI web app.

During testing, the platform matched human evaluators 85.1% of the time. It beats out other local baseline systems and stays highly competitive with expensive GPT-4 models, all while running securely on local hardware. We successfully pulled messy handwritten text, reconstructed code blocks, forced the AI to grade against strict rubrics, and handled concurrent uploads without crashing the server. This architecture demonstrates a practical, privacy-first path forward for educational technology.

B. FUTURE WORK

To push this architecture further, we recommend the following upgrades:

1) **Deploying a larger model:** Swapping in a 1.5B or 3B parameter model would likely close the gap on complex programming questions, pushing consistency well past 88%.
2) **GPU acceleration:** Running inference on a basic consumer GPU (like an RTX 3060) would slash evaluation times from 14.6 seconds to under 3 seconds per sheet.
3) **Diagram reading:** Adding a vision module to parse hand-drawn graphs and math equations.
4) **LMS integration:** Building an automated link to Canvas or Moodle so teachers can post grades directly to their existing digital grade books.
5) **Fully local OCR:** Replacing the Mistral API call with a locally hosted OCR model to make the system 100% offline and immune to network drops.

ACKNOWLEDGMENT

The authors would like to thank Prof. Indra R (Guide), Dr. Shambhavi B R (Head of Department), and the faculty of the Department of Computer Science and Engineering (Data Science) at B.M.S. College of Engineering, Bengaluru, for their invaluable guidance, constructive feedback, and unwavering support throughout this project. The authors also acknowledge the support of their institution for providing research facilities and access to digital resources used in this study.

REFERENCES

[1] A. Patil, S. Kulkarni, and N. Joshi, "Feedback-integrated automated grading and its longitudinal impact on student performance," IEEE Transactions on Learning Technologies, 2025.
[2] M. Li, T. Lv, L. Cui, Y. Lu, D. Florencio, C. Zhang, Z. Li, and F. Wei, "TrOCR: Transformer-based optical character recognition with pre-trained models," in Proc. AAAI, 2023, pp. 13094–13102.
[3] Microsoft Research, "Phi-3 technical report: A highly capable language model locally on your phone," arXiv preprint arXiv:2404.14219, 2024.
[4] L. Pecuchova, P. Novak, and M. Horak, "Generative AI for open-ended assessment grading in higher education," Computers & Education: Artificial Intelligence, vol. 8, 2025.
[5] J. Chen, H. Xu, and Y. Zhang, "4-bit post-training quantization for small language models: Efficiency without accuracy loss," Expert Systems with Applications, vol. 241, 2025.
[6] UNESCO, "Sustainable AI in education: Small language models and green deployment," UNESCO Digital Transformation Report, 2025.
[7] R. Thakur, P. Mehta, and A. Singh, "TrueGradeAI: Explainable automated grading via retrieval-augmented generation," IEEE Transactions on Education, 2025.
[8] A. Bernik, D. Kos, and T. Jakupovic, "Evaluating AI code assessment tools in programming examinations," Computers & Education, vol. 210, 2025.
[9] F. Mendonca, R. Lima, and A. Costa, "Prompt sensitivity in automated scoring of formative tasks," Journal of Educational Computing Research, 2025.
[10] B. Shi, X. Bai, and C. Yao, "An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition," IEEE Trans. Pattern Anal. Mach. Intell., vol. 39, no. 11, pp. 2298–2304, 2017.
[11] J. Cvengros, M. Blaho, and R. Horvath, "Optical character recognition for STEM examination scripts: Challenges and domain-specific fine-tuning," Expert Systems with Applications, vol. 237, 2025.
[12] Y. Huang, T. Lv, L. Cui, Y. Lu, and F. Wei, "LayoutLMv3: Pre-training for document AI with unified text and image masking," in Proc. ACM MM, 2022, pp. 4083–4091.
[13] W. Zhang, "Robustness of TrOCR under degraded scan conditions: A systematic stress-test," Pattern Recognition Letters, vol. 178, 2024.
[14] A. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, et al., "Mistral 7B," arXiv preprint arXiv:2310.06825, 2024.
[15] R. Kumar, A. Patel, and S. Nair, "Privacy architectures for locally deployed small language models: A systematic review," Applied Sciences, vol. 15, no. 3, 2025.
[16] Software Mind Research, "Benchmarking small language models on focused logical reasoning tasks," Technical Report, 2025.
[17] H. Park, J. Kim, and S. Lee, "Rubric is all you need: Chain-of-thought prompting with binary checkpoints for automated grading," arXiv preprint arXiv:2503.11842, 2025.
[18] S. Reddy, M. Kumar, and P. Rao, "AutoGrader+: Human-aligned automated scoring via action-verb rubric criteria," Computers & Education, vol. 208, 2025.
[19] J. Lee, H. Kim, and D. Park, "Human-in-the-loop verification for AI-graded open-ended assessments," IEEE Transactions on Learning Technologies, vol. 18, 2025.
[20] P. Agarwal, S. Jain, and A. Gupta, "Near-domain fine-tuning to mitigate solution-path bias in rubric-based AI grading," Journal of Artificial Intelligence in Education, 2025.
[21] M. Mohler, R. Bunescu, and R. Mihalcea, "Learning to grade short answer questions using semantic similarity measures and dependency graph alignments," in Proc. ACL-HLT, Portland, OR, 2011, pp. 752–762.
[22] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. EMNLP-IJCNLP, Hong Kong, 2019, pp. 3982–3992.
[23] S. Sharma and R. Verma, "RubricAI: Combining plagiarism detection with rubric-based logical evaluation for automated grading," International Journal for Multidisciplinary Research, vol. 7, no. 2, 2025.
[24] E. Emirtekin, "Fairness and bias in AI-based automated assessment systems: A systematic review," Educational Technology Research and Development, 2025.
[25] Qwen Team, Alibaba Cloud, "Qwen2.5 technical report," arXiv preprint arXiv:2412.15115, 2024.

[26] J. Kang, Y. Lee, and S. Park, "TPS-ResNet: Thin-plate spline rectification for unconstrained handwritten text recognition," Pattern Recognition, vol. 149, 2025.
[27] Discover AI Research Group, "Simulation-based grading for step-wise performance tasks in medical and STEM education," Discover AI, vol. 3, 2025.